

VOLLEY MARQUETTE — PROJECT REFERENCE

Document de référence — PDS & WBS • Solo project • Spring Boot • Next.js • Docker • CI/CD

1. Project Definition Sheet (PDS)

Champ	Détail
Nom du projet	Site web — Volley Club Marquette
Porteur	Nathan (développeur solo)
Objectif principal	Créer une application web complète (back + front + BDD) déployée en production, en maîtrisant des pratiques professionnelles : architecture en couches, Docker, CI/CD, JWT.
Objectifs pédagogiques	Maîtriser Spring Boot (API REST), JPA/Hibernate, Spring Security, Docker Compose, GitHub Actions, Next.js (TypeScript), déploiement cloud.
Public cible	Membres & supporters du Volley Club de Marquette
Contraintes	Projet solo • Budget zéro (outils gratuits uniquement) • Environnement Windows 11
Stack technique	Back : Java 21 + Spring Boot 3 Front : Next.js 14 + TypeScript + Tailwind CSS BDD : PostgreSQL ORM : Spring Data JPA / Hibernate Auth : Spring Security + JWT DevOps : Docker, Docker Compose, GitHub Actions Déploiement : Railway / Render
Critères de succès	API REST fonctionnelle et documentée (Swagger) Espace public + espace admin sécurisé Application conteneurisée et déployée Pipeline CI/CD opérationnel

2. Périmètre fonctionnel

Espace public (visible par tous)

Feature	Description	Entités liées
Accueil	Présentation du club + dernières actualités	Article
Actualités	Liste et détail des articles publiés	Article
Équipes	Liste des équipes du club (Séniors H/F, U18...)	Equipe
Joueurs	Fiche joueur rattachée à une équipe	Joueur, Equipe
Matches	Calendrier & résultats par équipe	Match, Equipe
Contact	Formulaire de contact simple	—

Espace admin (authentié)

Feature	Description	Accès
Login admin	Authentification → génération d'un token JWT	Public

Gestion actualités	Créer / modifier / supprimer un article	Admin
Gestion équipes	CRUD complet sur les équipes	Admin
Gestion joueurs	CRUD complet sur les joueurs	Admin
Gestion matchs	CRUD matchs + saisie des scores	Admin

3. Stack technique & architecture en couches

Couche	Techno	Rôle
API REST	Spring Boot 3 / Java 21	Exposition des endpoints HTTP
Sécurité	Spring Security + JWT	Authentification & autorisation
Service	@Service Spring	Logique métier
Repository (DAO)	Spring Data JPA	Accès base de données
ORM / Entités	Hibernate / JPA	Mapping objet-relationnel
Base de données	PostgreSQL	Persistance des données
Front-end	Next.js 14 + TypeScript	Interface utilisateur (SSR/SSG)
CSS	Tailwind CSS	Styling utilitaire sans Design System
Conteneurisation	Docker + Docker Compose	Isolation et orchestration des services
CI/CD	GitHub Actions	Build & tests automatiques à chaque push
Déploiement	Railway / Render	Hébergement cloud gratuit via Docker
Documentation API	Swagger / OpenAPI	Endpoints auto-documentés

4. Work Breakdown Structure (WBS)

Phase 0 — Mise en place de l'environnement	
ID	Tâche
0.1	Installer et vérifier Java 21, Docker, Node, Git
0.2	Créer et structurer le repo GitHub (branches, .gitignore, README)
0.3	Initialiser le projet Spring Boot via start.spring.io
0.4	Écrire le premier docker-compose.yml (PostgreSQL uniquement)
0.5	Vérifier la connexion Spring Boot ↔ PostgreSQL dans Docker

Phase 1 — API REST Spring Boot

ID	Tâche
1.1	Modéliser les entités métier : Joueur, Equipe, Match, Article
1.2	Écrire les classes @Entity (JPA) + migrations de schéma
1.3	Créer les Repository (Spring Data JPA = DAO)
1.4	Écrire les classes DTO (Data Transfer Object)
1.5	Implémenter les @Service (logique métier)
1.6	Créer les @RestController (endpoints GET/POST/PUT/DELETE)
1.7	Tester les endpoints avec Bruno / Postman
1.8	Configurer Spring Security + génération / validation JWT
1.9	Protéger les routes admin
1.10	Générer la documentation Swagger / OpenAPI
1.11	Écrire les tests unitaires des services (@SpringBootTest)

Phase 2 — Dockerisation complète

ID	Tâche
2.1	Écrire le Dockerfile multi-stage pour Spring Boot
2.2	Mettre à jour docker-compose.yml : back + postgres
2.3	Gérer les profils Spring (application-dev / application-prod)
2.4	Variables d'environnement et secrets (.env)

Phase 3 — Front-end Next.js

ID	Tâche
3.1	Initialiser le projet Next.js 14 (TypeScript + Tailwind)
3.2	Configurer la communication avec l'API Spring (axios/fetch)
3.3	Créer les pages publiques : Accueil, Actualités, Équipes, Matches
3.4	Implémenter la page de login (récupération + stockage du JWT)
3.5	Créer les pages admin CRUD protégées
3.6	Dockeriser le front (Dockerfile + intégration docker-compose)

Phase 4 — CI/CD & Déploiement

ID	Tâche
4.1	Créer le workflow GitHub Actions : build + tests à chaque push
4.2	Ajouter le lint et la vérification de format
4.3	Configurer les secrets GitHub (DB_URL, JWT_SECRET...)
4.4	Déployer sur Railway / Render via Docker
4.5	Configurer un nom de domaine (optionnel)
4.6	Vérifier le pipeline de bout en bout

5. Workflow Git recommandé

Branche	Rôle	Règle
main	Code stable, déployé en prod	Jamais de push direct — merge via PR uniquement
develop	Branche d'intégration	Base de travail au quotidien
feature/xxx	Nouvelle fonctionnalité	Créée depuis develop, mergée dans develop via PR
fix/xxx	Correction de bug	Même règle que feature/

Convention de commit : **type(scope): message** — ex : **feat(joueur): add POST endpoint**, **fix(auth): correct JWT expiration**

Ce document est vivant — mets-le à jour au fil du projet dans le dossier /docs de ton repo GitHub.